Customer Behavioral Segmentation System

Phase 1 Report — Synopsis

B.Tech Artificial Intelligence & Machine Learning

Field	Details
Project Title	Customer Behavioral Segmentation System
Subject	Problem-Based Learning (PBL) — AI/ML
Phase	Phase 1 — Data Collection & Exploratory Analysis
Algorithm(s)	K-Means Clustering, DBSCAN
Dataset	retail_finding_dataa.csv (1000 rows, 9 columns)
Language / Platform	Python 3.x | Jupyter Notebook | VS Code

1. Problem Description

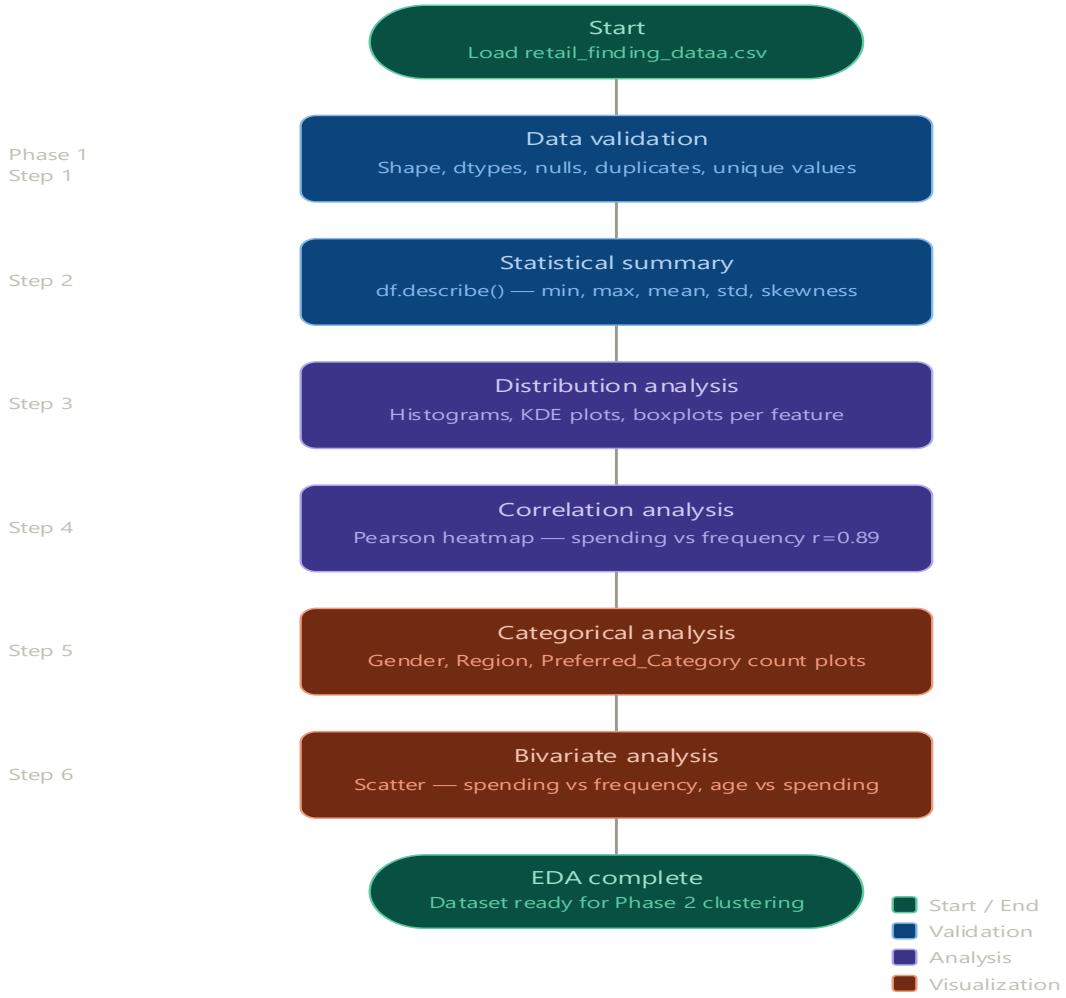
In today's highly competitive retail landscape, treating every customer identically is one of the most expensive mistakes a business can make. When the same promotional email goes to a premium buyer who has never touched a discount coupon and to a budget-driven shopper who only purchases during flash sales, the result is wasted marketing budget, irrelevant messaging, and ultimately, poor customer retention. This project directly addresses that gap by building a data-driven Customer Behavioral Segmentation System that groups customers based on how they actually behave — not just who they are on paper.

The core challenge is that customer behavior is multidimensional. Annual spending alone does not capture the full picture. A customer who spends ₹15,000 across 40 purchases behaves very differently from one who spends the same amount in just 3 high-ticket transactions. Similarly, a customer who uses discount coupons 60% of the time has fundamentally different price sensitivity than someone who rarely discounts at all. This project captures all of these nuances by bringing together five behavioral signals — Annual_Spending, Purchase_Frequency, Discount_Usage_Pct, Returns_Count, and Age — and applying unsupervised machine learning to find natural groupings within the data.

This is an unsupervised learning problem, meaning there are no pre-defined labels like 'good customer' or 'bad customer.' The algorithm discovers structure that already exists in the data. Two algorithms are used — K-Means clustering for clean, interpretable persona groups, and DBSCAN to additionally identify anomalous customers who do not fit neatly into any known segment (outlier detection). Together, they provide a 360-degree view of the customer base, enabling marketing teams to design campaigns that are relevant, cost-effective, and personalized.

In real-world business terms, this system answers the question: 'Who are my customers, how do they behave, and how should I talk to each group differently?' That is the fundamental promise of behavioral segmentation — and it is exactly the problem this project solves.

2. Steps of Implementation



The project is structured across three phases, each building on the previous:

1. Phase 1 — Data Collection & EDA: Load the retail dataset, perform structural validation (shape, dtypes, nulls, duplicates), analyze distributions through histograms and boxplots, compute skewness for all numerical features, and generate a correlation heatmap to identify relationships between variables. The goal is to deeply understand the data before any modeling begins.
2. Phase 2 — Preprocessing & K-Means Clustering: Extract behavioral features (Annual_Spending, Purchase_Frequency), apply StandardScaler to normalize the feature space, use the Elbow Method to determine the optimal number of clusters (k=4), fit K-Means with k-means++ initialization, assign business persona labels (Budget Buyers, Occasional Shoppers, Regular Spenders, Premium High-Value Customers), and perform extended clustering on demographic features (Age + Gender).
3. Phase 3 — Multi-Feature Training, DBSCAN & System Deployment: Train K-Means and DBSCAN across all five feature pair combinations, save all models, scalers, and label maps as pickle files, and deploy a menu-driven Python console application that loads any saved model, accepts live user inputs, scales them, and predicts the customer segment in real time.

3. Proposed Modules

The system is organized into the following functional modules, each serving a clear purpose in the overall pipeline:

Module	Description
Data Loading & Validation	Loads retail_finding_dataa.csv using Pandas. Checks shape, data types, null counts, duplicate records, and unique category values to confirm dataset readiness.
Exploratory Data Analysis (EDA)	Generates histograms, KDE plots, boxplots, scatter plots, and count plots to understand feature distributions and inter-variable relationships.
Correlation Analysis	Computes and visualizes the Pearson correlation matrix as a heatmap, identifying key behavioral signals (e.g., strong correlation between Annual_Spending and Purchase_Frequency).
Preprocessing & Scaling	Applies StandardScaler to bring all features onto a common scale, preventing high-magnitude variables from dominating distance calculations in K-Means.
K-Means Clustering	Uses the Elbow Method (WCSS over k=1 to 10) to find optimal k, fits KMeans with k-means++ initialization, and assigns cluster labels with Silhouette Score validation.
DBSCAN Clustering	Applies Density-Based Spatial Clustering to detect core regions and noise points. Uses k-distance graph to tune eps parameter. Identifies outlier customers as noise (label=-1).
Model Persistence	Saves trained KMeans models, DBSCAN models, StandardScalers, and cluster label maps to .pkl files using Python's pickle module for reuse in the deployment system.
Prediction & Deployment	Menu-driven console system (main.py) that loads saved pickle artifacts, accepts live feature inputs, applies the same scaler used during training, and outputs the predicted customer segment.

4. Required Topics from the Subject

4.1 Unsupervised Learning

This project is entirely built on unsupervised learning — the practice of finding hidden structure in data without any pre-assigned labels. Unlike classification where a model learns from labeled examples, K-Means and DBSCAN discover natural groupings on their own. The algorithm receives raw numerical data and partitions it into meaningful clusters based purely on similarity (proximity in feature space). Understanding unsupervised learning is central to this project because all business insights emerge from patterns the algorithm finds without human guidance.

4.2 Clustering Algorithms — K-Means & DBSCAN

K-Means is a centroid-based algorithm that partitions n data points into k clusters by iteratively minimizing the Within-Cluster Sum of Squares (WCSS). It begins by placing k centroids, assigns every point to its nearest centroid using Euclidean distance, then repositions centroids to the mean of their assigned points, repeating until convergence. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) takes a different approach — it identifies core points that have at least min_samples neighbors within a radius eps, expands clusters outward from those cores, and labels points that cannot reach any core as noise (label = -1). DBSCAN is particularly powerful for identifying outliers, which K-Means cannot do.

4.3 Feature Scaling & Preprocessing

StandardScaler is applied before clustering because K-Means uses Euclidean distance to measure similarity between data points. If Annual_Spending ranges from ₹50 to ₹20,000 while Purchase_Frequency ranges from 1 to 30, the spending dimension would completely dominate all distance calculations, making frequency irrelevant. StandardScaler converts each feature to have mean=0 and standard deviation=1, putting every feature on the same scale so each contributes equally to the distance metric. This preprocessing step is non-negotiable for any distance-based algorithm.

4.4 Evaluation Metrics — WCSS & Silhouette Score

WCSS (Within-Cluster Sum of Squares), also called inertia, measures the total squared distance from each data point to its cluster centroid. Lower WCSS means points are tightly packed around their centroids — better clusters. The Elbow Method plots WCSS against k values and looks for the 'elbow' — the point where adding more clusters gives diminishing improvement. The Silhouette Score complements this by measuring how similar each point is to its own cluster compared to neighboring clusters. A score close to +1 means well-separated, distinct clusters. Together, these metrics provide objective validation of cluster quality.

4.5 Data Visualization

This project uses seaborn and matplotlib extensively to communicate findings. Histograms and KDE plots reveal feature distributions and skewness. Boxplots expose outliers and spread. Scatter plots with color-coded clusters visually confirm segment separation. Heatmaps quantify feature correlations. Bar charts compare persona averages. Stacked bar charts show how personas distribute across age groups and genders. Visualization is not decorative here — every chart answers a specific analytical question about customer behavior.

4.6 Model Persistence with Pickle

Once models are trained, they need to be saved and reused without retraining. Python's pickle module serializes trained objects (KMeans model, DBSCAN model, StandardScaler, label maps) into .pkl binary files. When the deployment system (main.py) needs to make a prediction, it deserializes these files using pickle.load(), reconstructing the exact trained state of the model. This is how real-world ML systems work — training is a one-time step; serving predictions at runtime relies on saved artifacts.

5. Platform Required

Tool / Platform	Purpose	Reason for Use
Python 3.x	Core programming language	Rich ML ecosystem, clean syntax, standard in data science
Jupyter Notebook	Phase 1 & 2 development environment	Allows step-by-step cell execution with inline visualizations
VS Code	Phase 3 system development	Full IDE features for writing and debugging main.py
pandas	Data loading, manipulation, groupby	Industry-standard tabular data library in Python
NumPy	Numerical operations, distance calculations	Underpins all scientific computing in Python
scikit-learn	KMeans, DBSCAN, StandardScaler, metrics	Production-grade ML library with consistent API
matplotlib / seaborn	All visualizations	seaborn for statistical plots; matplotlib for full control
pickle	Model serialization and loading	Built-in Python module for saving/loading trained objects

6. Books and Link Sources

Books

- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow — Aurelien Geron (O'Reilly Media)
- Python Machine Learning — Sebastian Raschka & Vahid Mirjalili (Packt Publishing)
- Introduction to Machine Learning with Python — Andreas C. Muller & Sarah Guido (O'Reilly Media)
- Data Science from Scratch — Joel Grus (O'Reilly Media)

Online Resources

- scikit-learn Documentation — KMeans, DBSCAN, StandardScaler: https://scikit-learn.org/stable/
- GeeksforGeeks — K-Means Clustering Explained: https://www.geeksforgeeks.org/k-means-clustering-introduction/
- Towards Data Science — Customer Segmentation using K-Means: https://towardsdatascience.com/
- Stack Overflow — Python Clustering and Pickle: https://stackoverflow.com
- Kaggle — Retail Customer Segmentation Notebooks: https://www.kaggle.com/